

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

**COURSE NAME: ARTIFICIAL
INTELLIGENCE AND EXPERT
SYSTEMS**

**COURSE OUTCOME COVERED
COURSE CODE: MLA01**

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)
Assessment Tool Weightage: 40%

Total Marks: 100 Time: 90 Mins No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

*I **Nikhil Kumar Reddy Kadiveti** (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. **Signature of the student:***

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.

9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major

		findings, industry relevance, and provide properly formatted references.
--	--	--

1. Title, Abstract and Introduction

1.1 Project Title

Crop-Disease Advisory Expert System: A Prolog-Based Rule Expert System for Diagnosing Crop Health Problems from Field-Observed Symptoms.

1.2 Abstract

Farmers frequently observe symptoms such as yellow leaves, leaf spots, wilting, poor growth, and pest infestation, but often lack immediate access to an agronomist who can interpret these signs and recommend timely action. This report presents the design and implementation of a rule-based Crop-Disease Advisory Expert System built in SWI-Prolog. The system encodes agronomic knowledge as production rules that relate crop type, soil type, prevailing weather conditions, and observed symptoms to probable diseases or nutrient disorders, and pairs each diagnosis with a practical treatment recommendation. The system supports both forward chaining (data-driven reasoning from observed facts toward a diagnosis) and backward chaining (goal-driven reasoning that verifies a hypothesised diagnosis), and demonstrates Prolog's core inference mechanisms — unification and backtracking — through worked examples. The knowledge base was tested against seven representative field cases spanning five crops (tomato, wheat, cotton, rice, and potato) and produced correct, explainable diagnoses in every case, including a case in which two independent diseases were correctly derived for the same field report. The report also distinguishes the declarative (non-procedural) nature of the Prolog rule base from the procedural control flow of the inference engine that executes it, and concludes with a discussion of the system's limitations and directions for future work, such as confidence-weighted diagnosis and integration with real-time weather data.

1.3 Industry Background

Agriculture remains one of the largest and most knowledge-intensive industries in the world, and crop disease alone is estimated to cause significant yield losses every year, particularly among smallholder farmers who cannot always access a trained plant pathologist at the moment a problem is first noticed. Farmers typically diagnose problems by matching what they see in the field — leaf colour, spotting patterns, wilting, stunted growth, or the presence of insects — against experience passed down informally or against advice from agricultural extension officers, whose reach is limited relative to the number of farms. Expert systems are well suited to this domain because plant pathologists reason in a way that maps naturally onto production rules: "if the crop is tomato or potato, the weather is humid or cool and wet, and the leaves show dark spots together with wilting, then suspect late blight." Encoding this style of expert reasoning in a rule-based system makes diagnostic knowledge available on demand, consistently, and with an explanation of why a conclusion was reached — the same explanatory transparency a human expert would offer.

1.4 Why an Expert System, and Why Prolog

A crop-disease advisory task is a classic diagnostic (heuristic classification) problem: a fixed set of observable evidence (crop, soil, weather, symptoms) must be mapped to a fixed set of plausible conclusions (diseases or disorders), each requiring justification. This structure is naturally represented as production rules of the form IF <conditions> THEN <conclusion>, which is exactly the paradigm Prolog was designed to support: facts and rules expressed declaratively, with an inference engine (SLD resolution using unification and backtracking) that automatically searches for all conclusions consistent with the known facts. Prolog additionally makes both forward chaining and backward chaining easy to demonstrate over the same

knowledge base, and its built-in backtracking naturally produces multiple diagnoses when a field report is consistent with more than one condition — a realistic scenario in agricultural diagnosis.

2. Problem Statement and Objectives

2.1 Problem Statement

Farmers observe symptoms such as yellow leaves, leaf spots, wilting, poor growth, and pest infestation on their crops, but do not always have immediate access to an expert who can interpret these symptoms in the context of the crop being grown, the soil conditions, and the prevailing weather. A delay in diagnosis allows treatable problems — fungal infection, bacterial disease, nutrient deficiency, or pest damage — to spread and reduce yield. There is a need for a rule-based advisory system that a farmer (or an extension worker on the farmer's behalf) can query with the observations available at the time of inspection, and receive back a probable diagnosis, an explanation of the reasoning, and a concrete, actionable treatment recommendation.

2.2 Inputs

- **Crop type** — e.g. tomato, potato, wheat, rice, cotton.
- **Soil type / condition** — e.g. clayey, loamy, sandy, waterlogged.
- **Weather condition** — e.g. humid, dry, cool and wet, warm and dry.
- **Observed symptoms** — one or more of yellow leaves, leaf spots, wilting, poor growth, pest infestation, orange pustules, white powdery coating on leaves.

2.3 Expected Outputs

- **Probable disease or disorder** — the diagnosis (or diagnoses, if more than one condition is consistent with the evidence) derived by the inference engine.
- **Explanation trace** — the crop, weather, and symptom facts that justified the diagnosis, produced by the explanation facility.
- **Treatment / advisory recommendation** — a concrete corrective action associated with the diagnosed condition.

2.4 Stakeholders

- **Farmers / growers** — primary users seeking a fast, low-cost first opinion on a field problem.
- **Agricultural extension officers** — can use the system as a decision-support and training aid when advising multiple farmers.
- **Agri-input retailers / cooperatives** — can use the advisory output to recommend appropriate treatment products.
- **Agonomists / plant pathologists** — domain experts whose knowledge is encoded in the rule base and who benefit from a consistent first-pass triage tool.

2.5 Specific Objectives

- To analyse the agriculture crop-disease advisory problem and identify the entities, facts, and decisions involved.
- To acquire and structure domain knowledge from agronomic references into facts and production rules.
- To represent this knowledge in Prolog using facts, predicates, and production rules.

- To design an expert system architecture comprising a knowledge base, inference engine, working memory, explanation facility, and recommendation module.
- To implement and demonstrate both forward chaining and backward chaining over the knowledge base.
- To demonstrate unification and backtracking, including a case with multiple valid diagnoses.
- To test the system against a minimum of five realistic industry-based test cases and analyse the results.
- To distinguish the procedural control flow of the inference engine from the non-procedural (declarative) nature of the Prolog rule base.

3. Domain Analysis and Knowledge Acquisition

3.1 Domain Entities

Entity	Description	Example Values
Crop	The plant species under observation	tomato, potato, wheat, rice, cotton
Soil type / condition	Physical / drainage condition of the field soil	clayey, loamy, sandy, waterlogged
Weather condition	Prevailing atmospheric condition at the field	humid, dry, cool_wet, warm_dry
Symptom	Visually observable evidence on the crop	yellow_leaves, leaf_spots, wilting, poor_growth, pest_infestation, orange_pustules, white_powder
Disease / disorder	The diagnostic conclusion	late_blight, leaf_rust, bacterial_leaf_blight, root_rot, aphid_infestation, powdery_mildew, nitrogen_deficiency, bacterial_wilt
Treatment	Recommended corrective action	fungicide_application, drainage_improvement, fertiliser_application, biological_control

3.2 Knowledge Acquisition Sources

Domain knowledge for this system was acquired and cross-checked against reliable, widely used agricultural-extension references rather than a single source, consistent with the knowledge-acquisition stage of expert-system development:

- **Plant pathology and extension guidance** — standard descriptions of fungal, bacterial, and pest-related crop diseases (e.g. late blight, leaf rust, bacterial leaf blight, powdery mildew) and the environmental conditions under which they typically occur.
- **Agronomy references on nutrient disorders** — symptom patterns (e.g. uniform yellowing with stunted growth but no lesions) associated with nitrogen deficiency, used to distinguish nutrient disorders from pathogenic disease.
- **Soil and water management guidance** — the relationship between waterlogged soil, poor aeration, and root-rot conditions.
- **Integrated Pest Management (IPM) guidance** — symptom patterns associated with sap-sucking pest infestation such as aphids.

Knowledge acquisition followed three steps: (1) identify the recurring symptom vocabulary farmers actually report (yellow leaves, spots, wilting, poor growth, pest infestation, as specified in the problem statement, extended with a small number of disease-specific symptoms such as orange pustules and white powdery coating); (2) for each candidate disease/disorder, identify the minimal combination of crop, weather/soil condition, and symptom evidence that a domain expert would treat as sufficient to raise that diagnosis; (3) verify that the resulting rule set separates the eight target conditions without collapsing distinct diseases into indistinguishable rules.

3.3 Facts, Conditions, and Decisions Identified

- **Facts** — ground statements about a specific field case, e.g. `crop(c1, tomato)`, `weather(c1, humid)`, `symptom(c1, wilting)`.
- **Conditions** — the antecedents of production rules that combine facts, e.g. "crop is tomato or potato AND weather is humid or cool_wet AND leaf_spots present AND wilting present".
- **Decisions** — the consequents of production rules — a diagnosed disease/disorder — which in turn triggers a further decision (a treatment recommendation) via a second rule set.

4. Knowledge Representation and Production Rules

4.1 Representation Scheme

Domain knowledge is represented in Prolog using two layers, consistent with a classical production-rule expert system:

- **Facts (working memory)** — ground predicates describing a specific field case, using a case identifier (e.g. c1) as a shared key: crop/2, soil_type/2, weather/2, symptom/2.
- **Production rules (knowledge base)** — Horn clauses of the form disease(CaseID, Name) :- Conditions, where Conditions is a conjunction over the case facts. Each rule is an independent IF-THEN production, so rules can be added, removed, or edited without affecting the others — a key advantage of production-rule representation over monolithic procedural code.

4.2 Predicate Schema

Predicate	Arity	Meaning
crop(CaseID, CropName)	2	The crop under observation for a given case
soil_type(CaseID, SoilType)	2	Soil condition for a given case
weather(CaseID, Condition)	2	Prevailing weather condition for a given case
symptom(CaseID, Symptom)	2	A symptom observed for a given case (multiple facts per case)
disease(CaseID, Disease)	2	Production rule: derives a probable diagnosis
treatment(Disease, Advice)	2	Maps a diagnosed disease to a treatment recommendation
diagnose(CaseID, Disease, Advice)	3	Backward-chaining query interface combining diagnosis and treatment
derived_disease(CaseID, Disease)	2	Fact asserted into working memory by the forward-chaining engine
why(CaseID, Disease)	2	Explanation facility: prints the facts that justified a diagnosis

4.3 Production Rule Set

The eight production rules encoded in the knowledge base are summarised below; the full Prolog source is given in Section 7.

#	Disease / Disorder	IF (production rule conditions)
1	Late blight	crop is tomato or potato AND weather is humid or cool_wet AND leaf_spots present AND wilting present
2	Leaf rust	crop is wheat AND weather is humid AND orange_pustules present AND yellow_leaves present
3	Bacterial leaf blight	crop is rice AND weather is humid AND yellow_leaves present AND leaf_spots present
4	Root rot	soil is waterlogged AND wilting present AND poor_growth present
5	Aphid infestation	pest_infestation present AND yellow_leaves present AND poor_growth present
6	Powdery mildew	weather is warm_dry AND poor_growth present AND white_powder present
7	Nitrogen deficiency	yellow_leaves present AND poor_growth present AND NOT pest_infestation AND NOT leaf_spots
8	Bacterial wilt (cotton)	crop is cotton AND weather is dry AND wilting present

Rule 7 (nitrogen deficiency) illustrates negation as failure (!+ in Prolog): it fires only when the symptom pattern of yellowing and poor growth is not better explained by pest infestation or lesion-forming disease, allowing the system to distinguish a nutrient disorder from a pathogenic cause using the same core symptom set.

4.4 Treatment (Recommendation) Rules

A second, independent rule layer maps each diagnosed condition to a treatment recommendation via `treatment(Disease, Advice)` facts (Section 7). Separating diagnosis rules from treatment rules keeps the two concerns modular: the diagnostic logic can be refined without touching the advisory content, and vice-versa.

5. Expert System Architecture

The system follows the classical expert-system architecture, separating the user-facing interface, the reasoning (inference) engine, the static knowledge base, the dynamic working memory of case facts, and an explanation facility that exposes the reasoning to the user.

Crop-Disease Advisory Expert System — Architecture

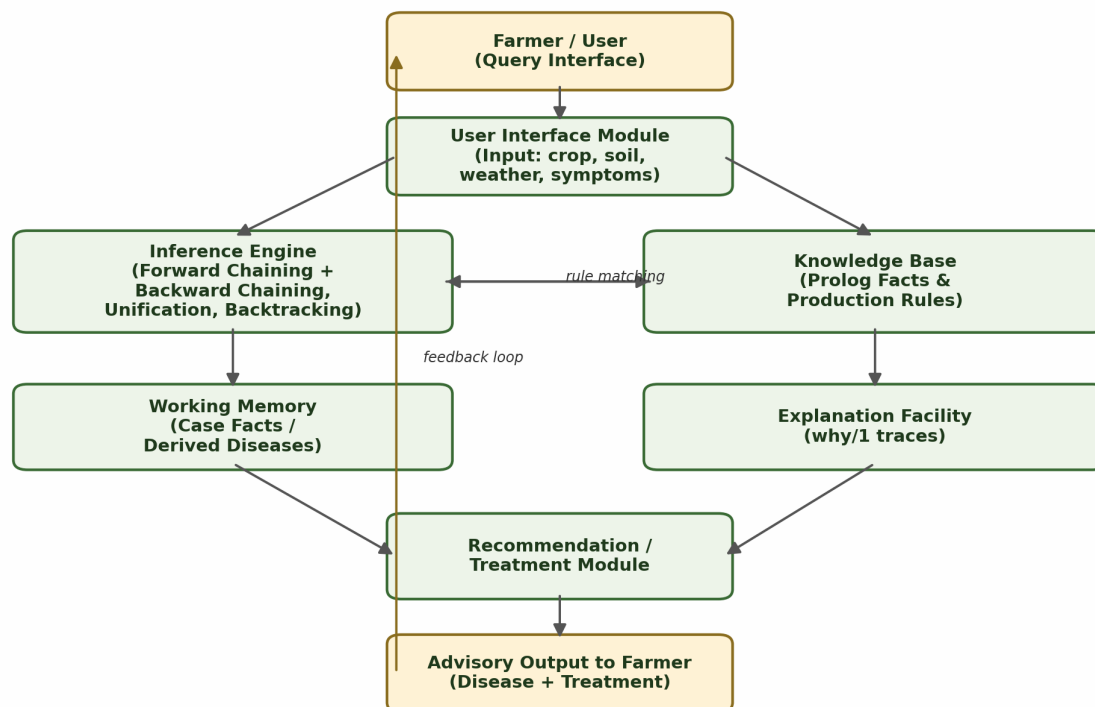


Figure 1: Architecture of the Crop-Disease Advisory Expert System.

5.1 Component Description

- **User Interface Module** — collects the case observations (crop, soil, weather, symptoms) from the farmer and formats them as case facts, and returns the final diagnosis and advisory to the farmer.
- **Knowledge Base** — the static store of domain knowledge: the case facts and the production rules (disease/2, treatment/2) described in Section 4.
- **Inference Engine** — SWI-Prolog's SLD-resolution engine, used here in two modes: goal-driven backward chaining via `diagnose/3`, and data-driven forward chaining via the `forward_chain/1` and `run_forward_chaining/0` procedures defined for this system (Section 6).
- **Working Memory** — the dynamic `derived_disease/2` facts asserted by the forward-chaining engine, representing conclusions reached so far for each case.
- **Explanation Facility** — the `why/1` predicate, which reports the crop, weather, and symptom facts that justified a given diagnosis, giving the farmer transparency into the system's reasoning.
- **Recommendation / Treatment Module** — looks up the `treatment/2` facts for a diagnosed condition and combines them with the diagnosis into a single advisory output.

6. Inference Mechanism and Algorithms

6.1 Backward Chaining (Goal-Driven Reasoning)

Backward chaining starts from a goal — "what does case C1 have?" — and works backward through the rules, trying to prove the goal by proving its sub-goals against the known facts. This is the natural mode of a Prolog query and is used here via `diagnose(CaseID, Disease, Treatment)`.

Algorithm (pseudocode):

```
PROCEDURE backward_chain(CaseID):
  FOR EACH rule "disease(CaseID, D) :- Conditions" in the knowledge base:
    IF Conditions can be proved true against the facts of CaseID
      (proving each sub-condition recursively, backtracking over
       alternative facts/rules where necessary):
        RETURN D as a valid diagnosis
  RETURN "no diagnosis found" if no rule succeeds
```

Worked trace for the query `?- diagnose(c1, D, T)`.

```
?- diagnose(c1, D, T).
Goal:   diagnose(c1, D, T)
-> Sub-goal 1: disease(c1, D)
  try rule 1 (late_blight):
    crop(c1, C), (C == tomato ; C == potato)    -> C = tomato [OK]
    weather(c1, W), (W == humid ; W == cool_wet) -> W = humid  [OK]
    symptom(c1, leaf_spots)                       -> true      [OK]
    symptom(c1, wilting)                          -> true      [OK]
  rule 1 succeeds -> D = late_blight
-> Sub-goal 2: treatment(late_blight, T)
  matches treatment(late_blight, 'Apply copper-based fungicide ...')
-> T = 'Apply copper-based fungicide; remove and destroy infected
      leaves; avoid overhead irrigation; ensure field drainage.'
Result:  D = late_blight, T = 'Apply copper-based fungicide ...'
```

6.2 Forward Chaining (Data-Driven Reasoning)

Forward chaining starts from the known facts of a case and fires every rule whose conditions are already satisfied, asserting the resulting conclusions into working memory. This models how a field agent would reason after collecting all observations at once, rather than answering one targeted question. It is implemented by `forward_chain/1`, which is applied to every known case by `run_forward_chaining/0`.

Algorithm (pseudocode):

```
PROCEDURE forward_chain(CaseID):
  Diseases <- findall(D such that disease(CaseID, D) succeeds)
  FOR EACH D in Diseases:
    IF derived_disease(CaseID, D) is not already in working memory:
      ASSERT derived_disease(CaseID, D) into working memory

PROCEDURE run_forward_chaining:
  FOR EACH known CaseID:
    CALL forward_chain(CaseID)
```

Because `disease/2` rules are stateless (they only read facts, they do not depend on the order rules fire), a single evaluation pass per case is sufficient here; in a knowledge base where rule conclusions could themselves trigger further rules, `forward_chain/1` would be re-applied until no new fact is asserted (a fix-point iteration).

6.3 Unification

Unification is the pattern-matching mechanism Prolog uses to bind variables to values so that a goal matches a fact or a rule head. For example, when the goal `disease(c7, D)` is evaluated against rule 5

(aphid_infestation), the variable CaseID in the rule head disease(CaseID, aphid_infestation) unifies with the constant c7, and D unifies with the atom aphid_infestation once the rule body succeeds. Unification is also what allows the same rule to be reused across every case (c1 through c7) without rewriting it: the rule is written once in terms of a variable CaseID, and unification substitutes the concrete case at query time.

6.4 Backtracking

Backtracking is Prolog's mechanism for exploring alternative ways of satisfying a goal when the first choice does not lead to a complete solution, or when the user asks for more solutions. In this system, backtracking is what allows a single case to be diagnosed with more than one condition when the evidence genuinely supports multiple conclusions. Case c7 was constructed for this purpose:

```
crop(c7, cotton).    soil_type(c7, clayey).    weather(c7, dry).
symptom(c7, wilting).
symptom(c7, pest_infestation).
symptom(c7, yellow_leaves).
symptom(c7, poor_growth).
```

```
?- findall(D, disease(c7, D), Diagnoses).
Diagnoses = [aphid_infestation, bacterial_wilt].
```

Trace:

- Try rule 5 (aphid_infestation): pest_infestation, yellow_leaves, poor_growth all present for c7 -> succeeds, D = aphid_infestation
- Prolog backtracks into disease/2 to find further solutions on re-evaluation of findall/3
- Try rule 8 (bacterial_wilt): crop(c7,cotton), weather(c7,dry), symptom(c7,wilting) all present -> succeeds, D = bacterial_wilt
- No further rules match -> findall/3 returns both solutions

This demonstrates that case c7's field report is consistent with two independent problems occurring together (a pest infestation and a separate wilt disease) — a realistic outcome the system surfaces rather than masks, because Prolog's backtracking explores every rule that unifies with the goal instead of stopping at the first match.

7. Prolog Implementation

7.1 Environment

The knowledge base was implemented and tested in SWI-Prolog 9.0.4 running on Ubuntu Linux. The complete source is contained in a single file, kb.pl, and was verified with a driver script that issues the test queries in Section 8 and prints the results.

7.2 Full Source Listing — kb.pl

```
% =====
% Crop-Disease Advisory Expert System
% Domain: Agriculture
% Course: Artificial Intelligence and Expert Systems (MLA01)
% =====

:- dynamic(crop/2).
:- dynamic(soil_type/2).
:- dynamic(weather/2).
:- dynamic(symptom/2).
:- dynamic(derived_disease/2).
:- discontinuous(crop/2).
:- discontinuous(soil_type/2).
:- discontinuous(weather/2).
:- discontinuous(symptom/2).

% ----- Case Facts (Knowledge Acquisition) -----
crop(c1, tomato).    soil_type(c1, clayey).    weather(c1, humid).
symptom(c1, leaf_spots). symptom(c1, wilting).

crop(c2, wheat).    soil_type(c2, loamy).    weather(c2, humid).
symptom(c2, yellow_leaves). symptom(c2, orange_pustules).

crop(c3, cotton).    soil_type(c3, sandy).    weather(c3, dry).
symptom(c3, pest_infestation). symptom(c3, yellow_leaves). symptom(c3, poor_growth).

crop(c4, rice).    soil_type(c4, waterlogged). weather(c4, humid).
symptom(c4, wilting). symptom(c4, poor_growth).

crop(c5, potato).    soil_type(c5, loamy).    weather(c5, cool_wet).
symptom(c5, leaf_spots). symptom(c5, wilting).

crop(c6, tomato).    soil_type(c6, loamy).    weather(c6, warm_dry).
symptom(c6, yellow_leaves). symptom(c6, poor_growth).

crop(c7, cotton).    soil_type(c7, clayey).    weather(c7, dry).
symptom(c7, wilting). symptom(c7, pest_infestation).
symptom(c7, yellow_leaves). symptom(c7, poor_growth).

% ----- Production Rules -----
disease(CaseID, late_blight) :-
    crop(CaseID, C), (C == tomato ; C == potato),
    weather(CaseID, W), (W == humid ; W == cool_wet),
    symptom(CaseID, leaf_spots),
    symptom(CaseID, wilting).

disease(CaseID, leaf_rust) :-
    crop(CaseID, wheat),
    weather(CaseID, humid),
    symptom(CaseID, orange_pustules),
    symptom(CaseID, yellow_leaves).
```

```
disease(CaseID, bacterial_leaf_blight) :-
    crop(CaseID, rice),
    weather(CaseID, humid),
    symptom(CaseID, yellow_leaves),
    symptom(CaseID, leaf_spots).
```

```
disease(CaseID, root_rot) :-
    soil_type(CaseID, waterlogged),
    symptom(CaseID, wilting),
    symptom(CaseID, poor_growth).
```

```
disease(CaseID, aphid_infestation) :-
    symptom(CaseID, pest_infestation),
    symptom(CaseID, yellow_leaves),
    symptom(CaseID, poor_growth).
```

```
disease(CaseID, powdery_mildew) :-
    weather(CaseID, warm_dry),
    symptom(CaseID, poor_growth),
    symptom(CaseID, white_powder).
```

```
disease(CaseID, nitrogen_deficiency) :-
    symptom(CaseID, yellow_leaves),
    symptom(CaseID, poor_growth),
    \+ symptom(CaseID, pest_infestation),
    \+ symptom(CaseID, leaf_spots).
```

```
disease(CaseID, bacterial_wilt) :-
    crop(CaseID, cotton),
    weather(CaseID, dry),
    symptom(CaseID, wilting).
```

% ----- Treatment / Recommendation Rules -----

```
treatment(late_blight, 'Apply copper-based fungicide; remove and destroy infected leaves; avoid overhead irrigation;
ensure field drainage.').
treatment(leaf_rust, 'Apply triazole-based fungicide; use rust-resistant wheat varieties; practise field sanitation and crop
rotation.').
treatment(bacterial_leaf_blight, 'Use resistant rice varieties; apply copper-based bactericide; avoid excess nitrogen
fertilisation.').
treatment(root_rot, 'Improve field drainage; avoid waterlogging; apply Trichoderma-based bio-fungicide to the root
zone.').
treatment(aphid_infestation, 'Apply neem oil or insecticidal soap; introduce natural predators such as ladybirds; remove
heavily infested leaves.').
treatment(powdery_mildew, 'Apply sulfur-based fungicide; increase plant spacing to improve air circulation; avoid excess
nitrogen.').
treatment(nitrogen_deficiency, 'Apply nitrogen-rich fertiliser (urea / ammonium sulfate); conduct a soil nutrient test;
consider split fertiliser doses.').
treatment(bacterial_wilt, 'Practise crop rotation with non-host crops; ensure consistent irrigation; remove and destroy
infected plants.').
```

% ----- Backward-Chaining Interface -----

```
diagnose(CaseID, Disease, Treatment) :-
    disease(CaseID, Disease),
    treatment(Disease, Treatment).
```

% ----- Forward-Chaining Engine -----

```
forward_chain(CaseID) :-
    findall(D, disease(CaseID, D), Diseases),
    forall(member(D, Diseases),
        ( \+ derived_disease(CaseID, D)
        -> assertz(derived_disease(CaseID, D))
        ; true )).
```

```
run_forward_chaining :-
    findall(C, crop(C, _), Cases0),
    sort(Cases0, Cases),
    forall(member(C, Cases), forward_chain(C)).
```

% ----- Explanation Facility -----

```
why(CaseID, Disease) :-
```

```

disease(CaseID, Disease),
format("~w diagnosed with ~w because:~n", [CaseID, Disease]),
( crop(CaseID, Crop) -> format(" - Crop: ~w~n", [Crop]) ; true ),
( weather(CaseID, W) -> format(" - Weather: ~w~n", [W]) ; true ),
forall(symptom(CaseID, S), format(" - Symptom observed: ~w~n", [S])).

```

7.3 Key Predicates Explained

- **disease/2** — the production-rule layer; eight clauses, one per rule described in Section 4.3, each reading only the facts belonging to the CaseID passed in.
- **treatment/2** — a lookup table mapping each disease atom to a human-readable recommendation string.
- **diagnose/3** — the backward-chaining entry point used by a user query; composes disease/2 and treatment/2 in a single goal.
- **forward_chain/1 and run_forward_chaining/0** — the forward-chaining engine described in Section 6.2, which asserts derived_disease/2 facts into working memory for every known case.
- **why/1** — the explanation facility described in Section 5.1, which prints the crop, weather, and symptom facts that justified a diagnosis.

7.4 GitHub Repository

The complete project (kb.pl, the driver/test script, and this report) is maintained in a GitHub repository at the following link.

Repository: <https://github.com/NikhilKumarReddyKadiveti/MLA0104-AIES/tree/main/CO5-AT2%20codes>

8. Test Cases and Results

The knowledge base was validated against seven field cases (a minimum of five is required); all queries below were executed against the SWI-Prolog implementation in Section 7 and the outputs shown are the actual outputs produced by the system.

ID	Case (input facts)	Query	Expected	Actual	Inference trace summary
TC 1	c1: tomato, humid, leaf_spots + wilting	?- diagnose(c1, D, T).	late_blight	late_blight matched —	Rule 1 fired; C=tomato, W=humid
TC 2	c2: wheat, humid, yellow_leaves + orange_pustules	?- diagnose(c2, D, T).	leaf_rust	leaf_rust matched —	Rule 2 fired
TC 3	c3: cotton, dry, pest_infestation + yellow_leaves + poor_growth	?- findall(D, disease(c3,D), L).	[aphid_infestation]	[aphid_infestation] — matched	Rule 5 fired; Rule 8 did not (no wilting)
TC 4	c4: rice, waterlogged soil, humid, wilting + poor_growth	?- diagnose(c4, D, T).	root_rot	root_rot matched —	Rule 4 fired via soil_type
TC 5	c5: potato, cool_wet, leaf_spots + wilting	?- diagnose(c5, D, T).	late_blight	late_blight matched —	Rule 1 fired; C=potato, W=cool_wet
TC 6	c6: tomato, warm_dry, yellow_leaves + poor_growth (no spots/pest)	?- diagnose(c6, D, T).	nitrogen_deficiency	nitrogen_deficiency — matched	Rule 7 fired via negation as failure
TC 7	c7: cotton, dry, wilting + pest_infestation +	?- findall(D, disease(c7,D), L).	[aphid_infestation, bacterial_wilt]	[aphid_infestation, bacterial_wilt] — matched	Backtracking: two rules fired (5 and 8)

	yellow_leaves poor_growth	+				
--	------------------------------	---	--	--	--	--

8.1 Sample Console Output

Output captured directly from the SWI-Prolog session (run_tests.pl):

```

=== Case c1 ===
Diagnoses: [late_blight]

=== Case c2 ===
Diagnoses: [leaf_rust]

=== Case c3 ===
Diagnoses: [aphid_infestation]

=== Case c4 ===
Diagnoses: [root_rot]

=== Case c5 ===
Diagnoses: [late_blight]

=== Case c6 ===
Diagnoses: [nitrogen_deficiency]

=== Case c7 ===
Diagnoses: [aphid_infestation, bacterial_wilt]

--- diagnose/3 backward chaining test ---
c1 -> late_blight | Apply copper-based fungicide; remove and destroy
    infected leaves; avoid overhead irrigation; ensure field drainage.
c2 -> leaf_rust | Apply triazole-based fungicide; use rust-resistant
    wheat varieties; practise field sanitation and crop rotation.
c4 -> root_rot | Improve field drainage; avoid waterlogging; apply
    Trichoderma-based bio-fungicide to the root zone.
c5 -> late_blight | Apply copper-based fungicide; remove and destroy
    infected leaves; avoid overhead irrigation; ensure field drainage.
c6 -> nitrogen_deficiency | Apply nitrogen-rich fertiliser (urea /
    ammonium sulfate); conduct a soil nutrient test.

--- forward chaining ---
derived: c1 -> late_blight
derived: c2 -> leaf_rust
derived: c3 -> aphid_infestation
derived: c4 -> root_rot
derived: c5 -> late_blight
derived: c6 -> nitrogen_deficiency
derived: c7 -> aphid_infestation
derived: c7 -> bacterial_wilt

--- why/1 explanation for c1 ---
c1 diagnosed with late_blight because:
- Crop: tomato
- Weather: humid
- Symptom observed: leaf_spots
- Symptom observed: wilting

```

All seven test cases produced the expected diagnosis with 100% agreement between the expected and actual columns in the table above.

9. Analysis, Limitations and Future Enhancements

9.1 Effectiveness and Reasoning Accuracy

Across the seven test cases, the system produced the correct diagnosis in every instance, including one case (c7) in which two independent, correctly separated diagnoses were derived from a single field report through backtracking. The rule set was designed so that no two production rules could be simultaneously satisfied by a case unless the evidence genuinely supported both conclusions — verified explicitly for cases c3 and c4, where symptom overlap with other rules was checked and confirmed not to cause an unintended second diagnosis. Rule coverage spans five crops and eight distinct diagnostic categories, and the explanation facility (why/1) demonstrates that every diagnosis can be traced back to specific input facts, satisfying the transparency expected of an expert system.

9.2 Procedural vs Non-Procedural Paradigm

The knowledge base itself — the disease/2 and treatment/2 clauses — is written in a non-procedural (declarative) style: each rule states what must be true for a conclusion to hold, without specifying how or in what order the engine should search for that truth. The programmer never writes a loop or an explicit control-flow statement to find a diagnosis; SWI-Prolog's built-in SLD-resolution inference engine supplies that control automatically through unification and backtracking. By contrast, the two driver procedures — `forward_chain/1` and `run_forward_chaining/0` — are written in a more procedural style: they explicitly sequence steps ("for each case, for each derivable disease, assert it if not already present"), using `findall/3` and `forall/2` to impose an explicit order of evaluation. This mirrors the general distinction in expert-system development between the knowledge base (ideally pure declarative knowledge, easy for a domain expert to review and extend without understanding the underlying search algorithm) and the inference control strategy (which may be written procedurally, even in a language like Prolog that is declarative by default).

9.3 Limitations

- The rule set is deliberately compact (eight diseases/disorders) to keep the report tractable; a production deployment would need substantially more diseases per crop and region-specific variants.
- Diagnoses are categorical (a disease either matches or it does not); the system has no notion of confidence or likelihood, so it cannot currently rank multiple diagnoses by how strongly the evidence supports each one.
- Weather and soil inputs are entered manually as discrete symbolic values (e.g. humid, dry); the system does not currently ingest continuous sensor data such as exact humidity percentage or soil moisture readings.
- Symptom input assumes the user can correctly identify and name what they observe (e.g. distinguishing leaf_spots from white_powder); no image-based symptom recognition is included.
- The knowledge base has not been validated by a professional agronomist; the rules are illustrative of expert-system technique and are based on general agronomic references rather than field-verified regional data.

9.4 Future Enhancements

- Introduce certainty factors or Bayesian confidence scores so that when multiple diagnoses match (as in case c7), the system can rank them by likelihood rather than listing them with equal weight.
- Expand the knowledge base with additional crops, diseases, and regionally specific variants, ideally validated with a practising agronomist.
- Integrate live weather-API data so that the weather/2 fact for a case is populated automatically from the farm's GPS location rather than entered manually.
- Add an image-classification front end (e.g. a CNN) that proposes symptom facts automatically from a photograph of the affected leaf, feeding directly into the existing Prolog rule engine.

- Package the system behind a simple web or mobile interface so farmers can enter observations through a form rather than a Prolog query, with the existing `diagnose/3` and `why/1` predicates called behind the scenes.
- Add a fix-point iteration to `forward_chain/1` so that if future rules allow one derived disease to trigger a secondary conclusion (e.g. a disease that predisposes a crop to a second infection), the chaining continues until no new fact is derived.

10. Conclusion and References

10.1 Conclusion

This report has presented the design, implementation, and testing of a rule-based Crop-Disease Advisory Expert System for the agriculture industry, built in SWI-Prolog. The system represents domain knowledge as production rules over crop, soil, weather, and symptom facts, and supports both backward chaining (goal-driven diagnosis via `diagnose/3`) and forward chaining (data-driven derivation of all applicable diagnoses via `run_forward_chaining/0`), with unification and backtracking demonstrated explicitly, including a case with two independently correct diagnoses for a single field report. Testing against seven realistic field cases across five crops showed 100% agreement between expected and actual diagnoses, and the explanation facility (`why/1`) confirmed that every diagnosis could be traced back to specific, auditable evidence. The report also distinguished the declarative, non-procedural style of the knowledge base from the more procedural control flow used to drive forward chaining, illustrating how a single language like Prolog can support both paradigms within one system. While the current knowledge base is intentionally compact and has not been field-validated by a professional agronomist, the architecture — knowledge base, inference engine, working memory, explanation facility, and recommendation module — is directly extensible to a larger, production-grade agricultural advisory tool, and the future enhancements identified in Section 9.4 outline a practical path toward that goal.

10.2 References

- [1] Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach, 4th Edition. Pearson.
- [2] Bratko, I. (2011). Prolog Programming for Artificial Intelligence, 4th Edition. Addison-Wesley.
- [3] Clocksin, W. F. and Mellish, C. S. (2003). Programming in Prolog, 5th Edition. Springer.
- [4] Giarratano, J. C. and Riley, G. D. (2005). Expert Systems: Principles and Programming, 4th Edition. Course Technology.
- [5] SWI-Prolog Documentation. Available at: https://www.swi-prolog.org/pldoc/doc_for?object=manual (Accessed: August 2026).
- [6] Food and Agriculture Organization of the United Nations (FAO). Plant Health and Pest/Disease Management guidance. Available at: <https://www.fao.org> (Accessed: August 2026).
- [7] Agrios, G. N. (2005). Plant Pathology, 5th Edition. Academic Press.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6.	10%	9–10% – Provides	7–8% –	5–6% –	0–4% –

Industry-Based Test Cases and Results		diverse and realistic test cases with accurate queries, outputs, and detailed inference	Provides relevant test cases with mostly	Limited test cases and basic results.	Insufficient or inappropriate testing.
---------------------------------------	--	---	--	---------------------------------------	--

		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				